Flying a Quadrotor Through a Known Environment Using RRT* Path Planning and Quintic Trajectories

Anshul Raje
Dept. of Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA 01609
Email: arraje@wpi.edu

Imaan Haider
Dept. of Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA 01609
Email: ihaider@wpi.edu

Zac Serocki
Dept. of Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA 01609
Email: zdserocki@wpi.edu

***Abstract*—This project presents an RRT* path planner and a quintic spline trajectory generator that navigate a quadrotor through known three-dimensional environments with cuboidal obstacles, first in simulation and then on a real Crazyflie. The planner found short collision-free paths around inflated obstacles, which were then simplified by greedy shortcutting. The generated waypoints were interpolated with a quintic spline that is continuous up to snap and starts and ends at rest, with segment durations scaled to respect velocity and acceleration limits. In the physics simulator, a cascaded PID controller tracked the trajectory without a detected collision on both provided maps, with a mean tracking error of 14 cm. On hardware, the Crazyflie closes its own attitude and velocity loops, so only an outer position controller was kept, which sent velocity commands to the Crazyflie as it flew through a course of physical boxes under Vicon motion capture. Its pose was placed inside a course-provided Gaussian splat reconstruction of the flight volume, so that collisions could be simulated without damaging the drone. The Crazyflie reached the goal in 5.8 s without a collision, tracking the planned path to within 3 cm on average, and lag in its velocity response caused a 15 cm overshoot at the goal before it settled.**

I. Problem Statement

To navigate a cluttered environment, a quadrotor must find a path around obstacles and convert it into motion that respects its physical limits. Because the quadrotor has a physical size and may not follow this planned path exactly, the path must leave enough clearance around obstacles. A path made of straight segments is also not directly flyable, as the sharp turns between segments would require sudden changes in velocity and cause the quadrotor to overshoot. The objective of this project was to navigate a quadrotor from a specified start position to a goal position in a known three-dimensional obstacle-rich environment, first in simulation and then on a real Crazyflie. This required four components: an environment reader to load the map and check for collisions, an RRT* planner to find a collision-free path, a trajectory generator to convert the path into smooth motion within velocity and acceleration limits, and a controller tuned to follow that trajectory.

II. Environment

In simulation, the quadrotor flew in a three-dimensional environment visualized with matplotlib. On hardware, the Crazyflie flew through a course of physical boxes, and its pose was placed inside a Gaussian splat reconstruction of the flight volume to simulate collisions. In both cases, each map was loaded from a text file containing the workspace boundary as well as the positions and dimensions of the obstacles. The start and goal positions were provided as part of the task. A safety margin was added around each obstacle to provide clearance for the quadrotor's body and allow for any small deviations during flight. The boundary was also shrunk by the same margin, so that the quadrotor kept the same clearance from the walls, floor, and ceiling of the workspace. These expanded obstacles and the shrunken boundary were also used to check whether points and paths were collision-free. To determine whether a point was in free space, we checked that it was inside the shrunken boundary, then iterated through all of the obstacles and checked whether the point was inside the inflated bounding box of any obstacle. To determine whether a line collides with the environment, we represented each obstacle as a convex hull defined by a collection of planes A. For a bounding box, A takes the form

$$A = \begin{pmatrix} I \\ -I \end{pmatrix}.$$

Thus, to determine whether a point is inside an obstacle, we check whether the following inequality holds:

$$\begin{pmatrix} I \\ -I \end{pmatrix} p \leq \begin{pmatrix} x_{max} \\ y_{max} \\ z_{max} \\ -x_{min} \\ -y_{min} \\ -z_{min} \end{pmatrix},$$

where I is the identity matrix of $\mathbb{R}^3$, which is multiplied by the point in question, and the result is compared to the extent of the hull. To represent the line, we use the form

$$p = (1-t)p_1 + tp_2, \quad t \in [0,1].$$

To determine whether the line intersects the convex hull, we check whether there exists a set of t for which the inequality is true. This is done by going term by term and shrinking the set of possible t. If a valid set remains at the end, this indicates a collision [1]. On the other hand, if the lower bound exceeds the upper bound, or if at any point the inequality does not hold,

the line does not collide with the hull. This method allows for collision detection with any obstacle that can be represented by a convex hull.

For the hardware flights, the map and the start and goal positions were provided through the course's map and task files, and the drone's state came from Vicon motion capture. Our planner, trajectory generator, and controller used only this map and the Vicon pose. During flight, the course software declared a virtual collision if the drone came within 0.10 m of the reconstructed geometry.

III. Path Planning

Before generating a trajectory, the quadrotor needs a collision-free path from the start position to the goal. A grid-based search would require the continuous three-dimensional workspace to be discretized, and the number of cells grows quickly with both the size of the map and the resolution. Sampling-based planners avoid this by exploring the free space with randomly chosen points and only checking for collisions where points are actually placed. The rapidly-exploring random tree (RRT) [2] grows a tree from the start position by repeatedly stepping from the tree toward a random sample, and will eventually find a path if one exists. However, each new point is simply attached to its nearest neighbor in the tree, so the resulting path is usually jagged and much longer than necessary, and there is no mechanism to shorten it once it is found. RRT* [3] keeps the same tree growth but adds two steps. Each new point is connected to whichever nearby point gives it the shortest path from the start, and the nearby points are then reconnected through the new point wherever that shortens their own paths. As a result, the path continues to improve as the tree grows, and converges to the optimal path as the number of samples increases. The subsections below describe a single iteration of the algorithm, followed by how the search terminates and how the final path is extracted. The parameter values given are those used for the simulation maps; the values used for the much smaller hardware volume are listed in Section VII-A.

A. Sampling

Each iteration begins by choosing a sample point for the tree to grow toward. This point was sampled uniformly within the map boundary, and resampled if it landed inside an obstacle. Purely uniform sampling explores the map evenly, but it only reaches the goal by chance, which can take many iterations in a large environment. To fix this, the goal itself was used as the sample 15% of the time. This is called goal biasing, and it pulls the tree toward the goal while the other samples keep exploring, so the tree can still find a way around obstacles. Once the goal was reached, the bias was turned off, as described in Section III-E.

B. Extending the Tree

Next, the node in the tree closest to the sample point was found by computing the Euclidean distance to every node. Instead of connecting straight to the sample, which could be far away, the tree only took a step of fixed length η toward it,

$$x_{\text{new}} = x_{\text{near}} + \eta \, \frac{x_{\text{rand}} - x_{\text{near}}}{\|x_{\text{rand}} - x_{\text{near}}\|} \tag{1}$$

where x_{near} is the nearest node and x_{rand} is the sample point. If the sample was closer than η, the new point was placed on the sample itself. A step size of $\eta = 1$ m was used. Keeping the steps short means each new edge covers only a small part of the map, so it is less likely to run into an obstacle, and the tree spreads out more evenly. The new point and the edge from x_{near} to it were then checked against the inflated obstacles. If either one was in collision, the point was thrown away and the next iteration began.

C. Parent Selection

In RRT, the new point would simply be connected to x_{near}. RRT* instead looks at every node within a radius r of the new point [3], called the near set X_{near}, and connects to whichever one gives the shortest path from the start. The cost $c(x)$ of a node is the length of the path from the start to that node through the tree. The parent was chosen as

$$x_{\text{parent}} = \underset{x \in X_{\text{near}}}{\arg\min} \left(c(x) + \|x_{\text{new}} - x\| \right) \tag{2}$$

where only nodes with a collision-free edge to x_{new} were considered. This check is needed because only the edge from x_{near} was checked in the previous step, and a cheaper parent could be on the other side of an obstacle. A search radius of $r = 2.5$ m was used. A larger radius finds better parents but checks more edges per iteration, which slows the planner down. Since r is larger than the step size, x_{near} is always in the near set, so there is always at least one valid parent. The new point was then added to the tree with cost

$$c(x_{\text{new}}) = c(x_{\text{parent}}) + \|x_{\text{new}} - x_{\text{parent}}\| \tag{3}$$

D. Rewiring

Parent selection finds the best way to reach the new point, but the new point may also give a shorter path to the nodes already around it. After x_{new} was added, each node x in the same near set X_{near} was checked to see if going through x_{new} would be cheaper than its current path,

$$c(x_{\text{new}}) + \|x - x_{\text{new}}\| < c(x) \tag{4}$$

If this was true and the edge between them was collision-free, x was disconnected from its old parent and reconnected to x_{new}. This is the step that lets RRT* keep improving paths after they are found, instead of being stuck with the first connection that was made.

Changing the parent of x lowers its cost by

$$\Delta = c(x_{\text{new}}) + \|x - x_{\text{new}}\| - c(x) \tag{5}$$

where $\Delta < 0$. Since the cost of every node is measured from the start, the costs of all of the descendants of x in the tree change by the same amount Δ. Updating only x would leave their costs out of date, and later parent selection and rewiring

steps would compare against the wrong values. So after each rewire, Δ was added to the cost of every node below x in the tree. This also means that if a goal node has already been found, rewiring anywhere along its branch lowers its cost as well.

E. Reaching the Goal and Termination

After each new node was added, the planner checked whether it was within a goal radius of 1 m of the goal. If it was, and the edge to the goal was collision-free, a goal node was attached to it as a child. The cost of this goal node is the length of the path found from the start to the goal.

The first path found is usually not the shortest one, since the tree has only just reached the goal. Instead of stopping there, the planner kept running for up to 500 more iterations so that rewiring could shorten the existing path and new branches could reach the goal by a different route. Each time another node reached the goal, a new goal node was added, and all of them were kept. The goal bias was also turned off at this point. The goal is now part of the tree, so a goal sample would just pick the goal node as the nearest node and add nothing useful. With the bias off, all of the remaining iterations went toward exploring and rewiring. Once the extra iterations were done, the goal node with the lowest cost was chosen as the final path. If the goal was never reached within 3000 iterations, the planner reported that it had failed.

F. Path Extraction and Simplification

Every node in the tree stores its parent, so the path was extracted by starting at the chosen goal node and following the parent links back to the start. Reversing this list gives the waypoints from start to goal.

Since the tree grows in steps of at most 1 m, this path contains many closely spaced waypoints with small zig-zags between them, left over from the random sampling. Many of these waypoints are not needed, as the quadrotor could fly straight past them without hitting anything. The path was simplified using greedy shortcutting, implemented as a sliding window over the waypoints. The window started at the first waypoint, and its end moved forward one waypoint at a time as long as the straight line between the start and end of the window was collision-free. When the check failed, the last waypoint that could be reached directly was kept, and the window was restarted from there. This was repeated until the window reached the goal. The same collision check against the inflated obstacles was used, so the simplified path is still collision-free. It is also never longer than the original, since each shortcut replaces a section of the path with a straight line. However, the shortcuts tend to pass close to the corners of the inflated obstacles, which is part of why the safety margin was needed. The simplified waypoints were then passed on for trajectory generation.

IV. Trajectory Generation

The waypoints from RRT* describe where the quadrotor should travel, but following straight-line segments between them could require abrupt changes in velocity at each turn. To give the controller a smooth trajectory, a quintic polynomial was generated between each pair of consecutive waypoints. The segments were joined using shared velocity and acceleration values, which were solved for so that the jerk and snap were also continuous, keeping the motion smooth through the waypoints. Time was assigned to each segment, then increased when necessary to keep the generated velocity and acceleration within their limits. The resulting trajectory provided position, velocity, and acceleration references for the controller.

A. Quintic Polynomial

For each coordinate of a segment, position was represented as a fifth-degree polynomial in the segment's local time t:

$$p(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5, \quad 0 \le t \le T. \tag{6}$$

Differentiating the position polynomial gives the desired velocity:

$$\dot{p}(t) = c_1 + 2c_2 t + 3c_3 t^2 + 4c_4 t^3 + 5c_5 t^4. \tag{7}$$

Differentiating again gives the desired acceleration:

$$\ddot{p}(t) = 2c_2 + 6c_3 t + 12c_4 t^2 + 20c_5 t^3. \tag{8}$$

The six coefficients were chosen to match the position, velocity, and acceleration at both ends of the segment. The starting conditions give $c_0 = p_0$, $c_1 = v_0$, and $c_2 = \frac{a_0}{2}$. Defining

$$P = p_f - p_0 - v_0 T - \frac{a_0 T^2}{2}, \tag{9}$$

$$V = v_f - v_0 - a_0 T, \tag{10}$$

$$A = a_f - a_0, \tag{11}$$

the remaining coefficients are

$$c_3 = \frac{10P}{T^3} - \frac{4V}{T^2} + \frac{A}{2T}, \tag{12}$$

$$c_4 = -\frac{15P}{T^4} + \frac{7V}{T^3} - \frac{A}{T^2}, \tag{13}$$

$$c_5 = \frac{6P}{T^5} - \frac{3V}{T^4} + \frac{A}{2T^3}. \tag{14}$$

Each coefficient has x, y, and z components. Once the coefficients were calculated, the polynomial was evaluated at times throughout the segment to obtain points along the three-dimensional curve. Its derivatives were evaluated at the same times to provide the velocity and acceleration references. Adjacent segments used the same position, velocity, and acceleration at their shared waypoint, allowing the trajectory to remain continuous as the quadrotor moved from one segment to the next. How the velocity and acceleration at each waypoint were chosen is described next.

B. Waypoint Velocities and Accelerations

The RRT* path gives the quadrotor points to pass through, but it does not specify how quickly to pass through them. Since each quintic segment is defined by the position, velocity, and acceleration at both of its ends, and neighboring segments share these values at their common waypoint, the velocity and acceleration are continuous no matter which values are chosen. The velocity and acceleration at the start and goal were set to zero so that the quadrotor starts and ends at rest. For a path with n waypoints, this leaves the velocities v_i and accelerations a_i at the $m = n - 2$ intermediate waypoints to be chosen.

These values were chosen so that the next two derivatives, jerk and snap, are also continuous at every intermediate waypoint. A jump in jerk would require the quadrotor to change its tilt rate instantly, which it cannot do, so the controller would fall behind the reference. With jerk and snap continuous, the resulting spline is also the one that minimizes the integral of the squared jerk over the whole trajectory for the given segment times.

Differentiating the acceleration of the quintic gives the jerk and snap:

$$\dddot{p}(t) = 6c_3 + 24c_4 t + 60c_5 t^2, \tag{15}$$

$$\ddddot{p}(t) = 24c_4 + 120c_5 t. \tag{16}$$

Substituting c_3, c_4, and c_5 from the previous subsection, the jerk and snap at the two ends of a segment of duration T can be written in terms of its boundary values, where $\Delta = p_f - p_0$:

$$\begin{aligned}
\dddot{p}(0) &= \tfrac{60\Delta}{T^3} - \tfrac{36v_0+24v_f}{T^2} - \tfrac{9a_0-3a_f}{T}, \\
\dddot{p}(T) &= \tfrac{60\Delta}{T^3} - \tfrac{24v_0+36v_f}{T^2} - \tfrac{3a_0-9a_f}{T}, \\
\ddddot{p}(0) &= -\tfrac{360\Delta}{T^4} + \tfrac{192v_0+168v_f}{T^3} + \tfrac{36a_0-24a_f}{T^2}, \\
\ddddot{p}(T) &= \tfrac{360\Delta}{T^4} - \tfrac{168v_0+192v_f}{T^3} - \tfrac{24a_0-36a_f}{T^2}.
\end{aligned} \tag{17}$$

At an intermediate waypoint p_i, the jerk and snap at the end of the segment before it were set equal to the jerk and snap at the start of the segment after it. Writing $\Delta_i = p_{i+1} - p_i$ and moving the known positions to the right-hand side, the jerk condition becomes

$$\begin{aligned}
&-\tfrac{24}{T_{i-1}^2} v_{i-1} - \tfrac{3}{T_{i-1}} a_{i-1} + 36\left(\tfrac{1}{T_i^2} - \tfrac{1}{T_{i-1}^2}\right) v_i \\
&+ 9\left(\tfrac{1}{T_{i-1}} + \tfrac{1}{T_i}\right) a_i + \tfrac{24}{T_i^2} v_{i+1} - \tfrac{3}{T_i} a_{i+1} \\
&= 60\left(\tfrac{\Delta_i}{T_i^3} - \tfrac{\Delta_{i-1}}{T_{i-1}^3}\right),
\end{aligned} \tag{18}$$

and the snap condition becomes

$$\begin{aligned}
&\tfrac{168}{T_{i-1}^3} v_{i-1} + \tfrac{24}{T_{i-1}^2} a_{i-1} + 192\left(\tfrac{1}{T_{i-1}^3} + \tfrac{1}{T_i^3}\right) v_i \\
&+ 36\left(\tfrac{1}{T_i^2} - \tfrac{1}{T_{i-1}^2}\right) a_i + \tfrac{168}{T_i^3} v_{i+1} - \tfrac{24}{T_i^2} a_{i+1} \\
&= 360\left(\tfrac{\Delta_{i-1}}{T_{i-1}^4} + \tfrac{\Delta_i}{T_i^4}\right).
\end{aligned} \tag{19}$$

Here, T_{i-1} and T_i are the durations of the segments before and after waypoint i, respectively. Terms involving the start or goal drop out, since their velocity and acceleration are zero. Each intermediate waypoint contributes two unknowns and two equations, giving a square linear system of size $2m$ with unknowns ordered as $[v_1, a_1, \ldots, v_m, a_m]$. Each equation only involves a waypoint and its two neighbors, so the matrix is block tridiagonal with 2×2 blocks. The matrix depends only on the segment durations, so the x, y, and z components were solved together as three right-hand sides of the same system. The solved v_i and a_i were then used at the end of one segment and the beginning of the next.

C. Spline Collision Repair

The simplified path is collision-free, but the spline through it can bulge outward between waypoints and clip an inflated obstacle, since the shortcuts already pass close to obstacle corners. To catch this, each segment of the spline was sampled at 50 points using the segment durations from Section IV-D, and the line between each pair of consecutive samples was checked for collisions. If a segment collided, the midpoint of its two waypoints was inserted as a new waypoint and the spline was solved again. The extra waypoint pulls the curve closer to the straight, collision-free segment. This was repeated until the spline was collision-free, and trajectory generation failed if it still collided after 30 insertions.

D. Time Allocation and Motion Limits

A smooth, collision-free curve still needs enough time for the quadrotor to follow it without exceeding the selected velocity and acceleration limits. The time assigned to each segment determines how quickly the quadrotor moves along that part of the trajectory. Each segment was first assigned a duration based on the distance between its waypoints and the velocity and acceleration limits:

$$T_i = \max\left(\frac{d_i}{v_{\max}},\ 2\sqrt{\frac{d_i}{a_{\max}}}\right), \quad d_i = \|p_{i+1} - p_i\| \tag{20}$$

where $v_{\max}$ and $a_{\max}$ are the selected velocity and acceleration limits. The first term is the time to cover the segment at the maximum velocity. The second is the time to cover it by accelerating at $a_{\max}$ for the first half and decelerating for the second half, which dominates for short segments. After generating the quintic segments, the velocity and acceleration magnitudes were sampled at 500 points along each segment to estimate their peak values, v_{peak} and a_{peak}. The selected trajectory limits were $v_{\max} = 6\,\text{m/s}$ and $a_{\max} = 3\,\text{m/s}^2$ in simulation, and $v_{\max} = 1.5\,\text{m/s}$ and $a_{\max} = 1.25\,\text{m/s}^2$ on hardware. A scaling factor was then calculated to determine whether the trajectory needed more time:

$$k = \max\left(1,\ \frac{v_{\text{peak}}}{v_{\max}},\ \sqrt{\frac{a_{\text{peak}}}{a_{\max}}}\right). \tag{21}$$

If either peak exceeded its limit, k was greater than one. The trajectory timestamps were multiplied by k, while the velocity and acceleration references were divided by k and k^2, respectively. This increased the time available to follow the same curve and reduced the required speed and acceleration.

V. Controller

The planned trajectory provides the quadrotor with desired positions, velocities, and accelerations, but it does not make the quadrotor follow them on its own. The controller compares the desired motion with the quadrotor's current state and calculates control inputs to reduce the difference. In the physics simulator, the full cascaded controller was implemented, with a position loop feeding a velocity loop that computes the motor commands. On hardware, only the position loop was kept, as described in Section V-B. Its gains determine how strongly the quadrotor responds to tracking errors. These gains must be chosen carefully because a response that is too weak can leave the quadrotor behind the trajectory, while an overly aggressive response can cause overshoot.

A. Gain Tuning for Simulation

The position and velocity gains were adjusted through simulation tests. Runs were compared using the time required to reach the goal, average and maximum position tracking errors, and collision outcomes. Motor commands, as well as velocity and acceleration plots over time, were also examined to identify abrupt changes, oscillations, and motor saturation. Fixed seeds were used to keep the planned path the same, making it easier to compare the effect of gain adjustments.

The selected position and velocity gains are shown below.

TABLE I: Selected position and velocity controller gains

Controller	Axes	K_p	K_i	K_d
Position	x, y, z	0.25	0.0	0.0525
Velocity	x, y, z	2.25	0.0	0.05

The same gains were used across all three axes and on both maps. These gains were tested with trajectory velocity and acceleration limits of $6\,\mathrm{m/s}$ and $3\,\mathrm{m/s^2}$, respectively. These parameters were selected because they provided closer trajectory tracking while keeping the flight times relatively short.

B. Policy Implementation for Hardware

To run the controller on the drone, we implemented the provided Policy interface in a TrajectoryPolicy class. The hardware loop calls the policy's act() method at 15 Hz. Each call receives the drone's current state and returns a velocity command in the body frame. Unlike the simulated quadrotor, which took motor commands, the Crazyflie takes a velocity command and closes the attitude and velocity loops in its own firmware. The policy therefore acts only as an outer position loop, and the velocity loop and its gains from simulation were dropped. The hardware gains could not be carried over from simulation either, since the simulated velocity loop and the Crazyflie's firmware respond differently, so they were tuned in flight, as described in Section VII-B. The policy loads the waypoints and velocities produced by the trajectory generator, so the controller can use the planned velocities directly.

1) Target Selection: The target is chosen using pure pursuit [4]. Instead of steering toward the closest point on the path, the drone steers toward a point slightly ahead of it. The policy keeps an index i into the waypoint list and advances it while the next waypoint is closer to the drone than the current one, which gives the closest point on the path. From that point, the policy walks forward along the path segments by a lookahead distance of $L = 0.3\,\mathrm{m}$ and linearly interpolates within the segment where the remaining distance runs out. This lookahead point is the target $\mathbf{p}_{\text{target}}$. If the lookahead runs past the end of the path, the goal itself is used. Aiming ahead of the drone keeps it making forward progress, whereas steering toward the closest point would make it stall on the path.

2) Control Law: The commanded world-frame velocity is the planned velocity at the closest waypoint, used as a feedforward term, plus a proportional correction toward the path:

$$\mathbf{v}_{\text{cmd}} = \mathbf{v}_{\text{ff}} + K_p\,\mathbf{e}_{\perp}, \qquad \mathbf{v}_{\text{ff}} = \mathbf{v}_i. \tag{22}$$

The error $\mathbf{e} = \mathbf{p}_{\text{target}} - \mathbf{p}$ points from the drone to the lookahead point. Its component along the direction of travel $\hat{\mathbf{u}} = \mathbf{v}_{\text{ff}}/\|\mathbf{v}_{\text{ff}}\|$ was removed,

$$\mathbf{e}_{\perp} = \mathbf{e} - (\mathbf{e}\cdot\hat{\mathbf{u}})\,\hat{\mathbf{u}}, \tag{23}$$

so that the feedback only pulls the drone back onto the path, while the feedforward alone sets its speed along it. Without this projection, the lookahead point, which is always 0.3 m ahead, would add a constant forward push and the drone would fly faster than the plan. When the planned speed is close to zero ($\|\mathbf{v}_{\text{ff}}\| < 10^{-3}$ m/s), at the start and end of the path, the full error $\mathbf{e}$ was used instead so that the drone still settles on the goal. A gain of $K_p = 2.5\ \mathrm{s^{-1}}$ was used. The magnitude of $\mathbf{v}_{\text{cmd}}$ was capped at 1.5 m/s, matching the trajectory's velocity limit, so the cap does not flatten the planned speed profile. The command is finally rotated from the world frame into the body frame using the drone's yaw ψ,

$$\mathbf{v}_{\text{body}} = \begin{pmatrix} \cos\psi & \sin\psi & 0 \\ -\sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \mathbf{v}_{\text{cmd}}, \tag{24}$$

with a commanded yaw rate of zero. On the first call, the policy also checks that the drone is within 0.5 m of the start of the path, and refuses to fly otherwise, since a large initial offset means that the Vicon-to-scene transform is wrong.

VI. Results: Simulation

The complete pipeline was first evaluated in a physics simulator of the quadrotor, using the cascaded PID controller described above, on two of the provided simulation maps. These maps are larger than the hardware volume and were used to develop and tune the planner and trajectory generator before they were moved to the drone. Map 1 is a $10\times25\times6$ m room with a blocked window and a raised ledge blocking the goal. The quadrotor has to thread through the window in the barrier and maneuver over the ledge to reach the goal. Map 4 is an open $45\times35\times6$ m workspace containing a small cube

TABLE II: Results over ten random seeds per map (mean $\pm$ std. dev.).

	Map 1	Map 4
Success Rate	10/10	10/10
RRT* Tree nodes	846 ± 102	662 ± 32
Raw tree path length (m)	22.97 ± 0.57	49.21 ± 3.31
Simplified path length (m)	22.42 ± 0.60	45.40 ± 0.82
Simplified waypoints	6.6 ± 0.8	3.1 ± 0.3
Flight time (s)	12.8 ± 1.6	12.0 ± 0.7
Mean position error (m)	0.137 ± 0.040	0.134 ± 0.010
RMS position error (m)	0.156 ± 0.041	0.141 ± 0.012
Max position error (m)	0.287 ± 0.050	0.189 ± 0.026

and a single full-height wall between the start and goal. This map tests navigation in a large environment at high speed. For these experiments, we inflated the boundaries by 0.5 m for the planner's collision checking, and used a step size of 1 m for RRT*, a search radius of 2.5 m, a goal bias of 15%, and a goal radius of 1 m. A flight counted as reaching the goal once the quadrotor came within 0.8 m of it. Since RRT* is non-deterministic, we ran the trial on each map 10 times and recorded the results in Table II.

A. Path Planning

The path planner was able to successfully generate a path to the goal on every trial. For Map 1, it required 846 nodes to reach the goal, with a standard deviation of 102 nodes, which is far more than Map 4, which only needed 662 nodes with a standard deviation of 32 nodes. Despite Map 4 being much larger than Map 1, it required fewer nodes because the environment does not have many obstacles to plan around. In contrast, on Map 1 the planner has to plan through the window, where many connections are rejected if they land on the other side of the barrier. The explored trees for Maps 1 and 4 can be seen in Figures 1 and 2, respectively. Once the goal was reached, an extra 500 points were added to optimize the path, resulting in an average path length of 22.97 m with a standard deviation of 0.57 m on Map 1, and an average length of 49.21 m with a standard deviation of 3.31 m on Map 4. This small variance in the path length hints that we are substantially optimizing the path. We were then able to prune closely spaced nodes to achieve a final path that consisted of an average of 6.6 nodes for Map 1 and 3.1 nodes for Map 4, with minimal variation for each, making the paths easier to fly. The simplified waypoints and the resulting splines are shown in Figures 10 and 11.

B. Map 1: Trajectory Tracking Performance

Using the selected controller gains and trajectory limits of $6\,\mathrm{m/s}$ and $3\,\mathrm{m/s^2}$, the quadrotor reached the goal in $10.60\,\mathrm{s}$ without a detected collision. Figure 12 shows an executed flight path, position and velocity histories, motor commands, tracking error, and distance to the goal. The average position tracking error in this run was $0.135\,\mathrm{m}$, with a maximum of $0.233\,\mathrm{m}$. The tracking error of the flights can be seen in Figure 3, where it deviates from the target but stays within the safety margin bounds. The position and velocity tracking

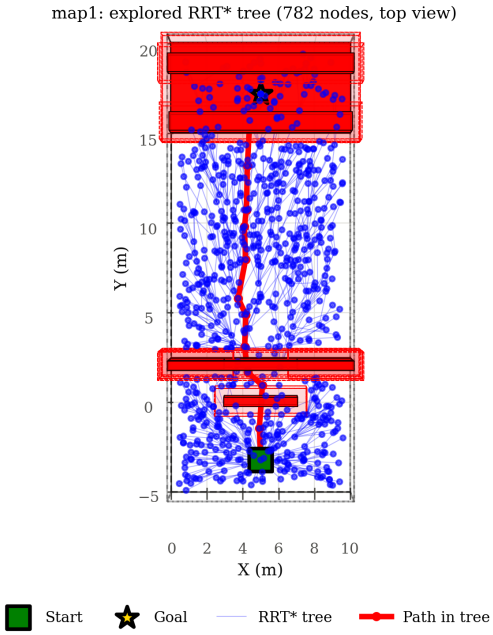


Fig. 1: Top-down view of the RRT* nodes expanded on Map 1.

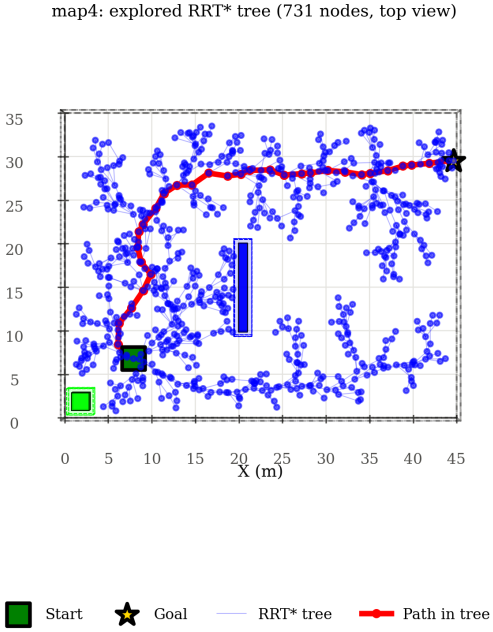


Fig. 2: Top-down view of the RRT* nodes expanded on Map 4.

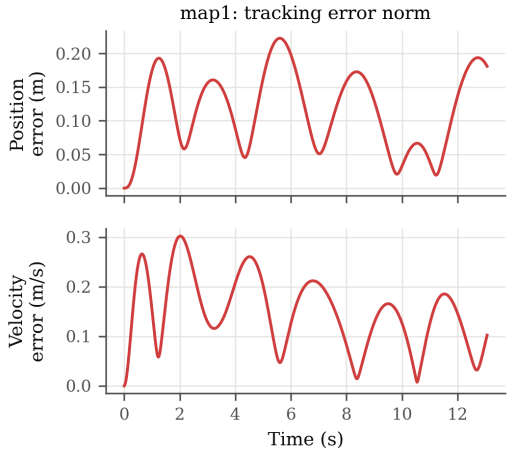


Fig. 3: Map 1 error vs. time. The error stays bounded within the safety region while maintaining performance.

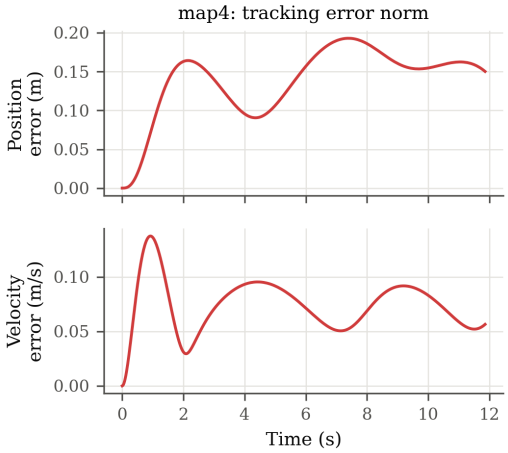


Fig. 4: Map 4 error vs. time. The error stays bounded within the safety region while maintaining performance.

on each axis can be seen in Figure 8. The position tracks the desired trajectory quite well, except for minor deviations in the x direction, while there is a good bit of overshoot in the velocity.

C. Map 4: Trajectory Tracking Performance

The same controller gains and trajectory limits were used on Map 4. The quadrotor reached the goal tolerance in $11.80\,\mathrm{s}$ without a detected collision. Figure 13 shows an executed flight path, position and velocity histories, motor commands, position tracking error, and distance to the goal. The average position tracking error was $0.125\,\mathrm{m}$, with a maximum of $0.192\,\mathrm{m}$. The tracking error of the flights can be seen in Figure 4, where it deviates from the target less than on Map 1 and stays within the safety margin bounds. The position and velocity tracking on each axis can be seen in Figure 9. The position tracks the desired trajectory very closely, with minimal overshoot in the velocity.

VII. Results: Hardware

A. Setup

The Crazyflie's pose came from Vicon, and the policy's velocity commands were sent to it by radio. The planner parameters used for the simulation maps were too coarse for a volume only 2.4 m wide, where a 1 m step would jump across most of the gaps between boxes. They were therefore reduced to a step size of 0.35 m, a goal radius of 0.35 m, and a search radius of 0.8 m. Because the smaller steps need more iterations to settle, the planner was also allowed 1500 extra iterations after first reaching the goal instead of 500. The obstacles were inflated by 0.10 m, and the trajectory limits were $v_{\max} = 1.5$ m/s and $a_{\max} = 1.25$ m/s^2.

For the final flights, RRT* returned a path with a single intermediate waypoint, at $(3.70, 0.14, 1.00)$ m, which passes between the first two boxes and to the right of the third (Fig. 6). The spline through it is 5.17 m long, only 1.3% longer than the straight line between the start and goal, and lasts 6.44 s. It reaches the 1.5 m/s velocity limit, with a peak acceleration of 0.74 m/s^2, so on this path the velocity limit set the duration. The trajectory's closest approach to the reconstruction was 0.267 m, well above the required 0.10 m.

B. Tracking Performance

Over eleven flights, the feedback gain was steadily raised to improve tracking, and the trajectory's velocity and acceleration limits were raised from 0.6 m/s and 1.0 m/s^2 to shorten the flight, ultimately landing on $K_p = 2.5$, $v_{\max} = 1.5$ m/s, and $a_{\max} = 1.25$ m/s^2. The rest of this section analyzes flight 11, the final flight with these settings.

Table III summarizes flight 11 over two windows: the approach, from takeoff until the drone first came within 0.15 m of the goal at $t = 5.80$ s, and the full logged run of 12.3 s, which also includes the overshoot at the goal and the hover afterward. The off-path error is the distance from the drone to the closest point on the planned spline, and the clearance is the distance from the drone to the nearest reconstructed surface, read from the course-provided ESDF. The flown path is shown in 3D in Fig. 5 and from the top and side in Fig. 6, and the clearance, off-path error, speed, height, and divergence over the whole run are shown in Fig. 7.

The drone reached the goal in 5.80 s without a collision, slightly ahead of the 6.44 s plan. It might arrive early because the run counts as reaching the goal within 0.15 m of it, and the lookahead lets it start the final approach before the plan does. It peaked at 1.42 m/s, just under the 1.5 m/s limit, so the policy's speed cap was never reached.

The drone stayed within 4 cm of the path for the first 4 s, while the path is nearly straight; most of this error is the 4 cm offset it started with at takeoff, which the feedback removed within 0.7 s. Almost all of the remaining error during the

TABLE III: Hardware flight 11 on the training task ($K_p = 2.5$). The approach ends when the drone first came within 0.15 m of the goal; the full run includes the overshoot and the hover at the goal.

	Approach (0–5.8 s)	Full run (0–12.3 s)
Time to goal (planned: 6.44 s)	5.80 s	–
Peak speed (limit: 1.5 m/s)	1.42 m/s	1.42 m/s
Mean / RMS off-path error	2.9 / 3.3 cm	3.0 / 4.2 cm
Max off-path error	6.3 cm	14.9 cm
Mean |error| in x / y / z	0.4 / 2.5 / 1.0 cm	–
Min clearance (limit: 10 cm)	31.6 cm	31.6 cm
Mean / max divergence	0.21 / 0.49 m/s	0.17 / 0.70 m/s
Final distance to goal	–	0.9 cm
Median pose age / latency	0.9 / 5.4 ms	
Collision	None	

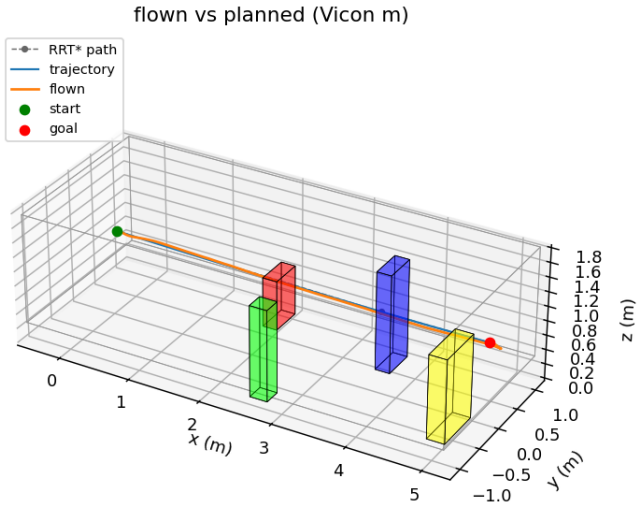


Fig. 5: Flight 11 in 3D, in the Vicon frame: the RRT* path, the planned trajectory, and the flown path through the four boxes.

approach is in y, the direction in which the path curves. The path drifts toward $-y$ until it passes the third box and then turns back toward $+y$ to reach the goal. In this final turn, the drone swung wide to the outside of the curve, on the $-y$ side, as seen in the top view of Fig. 6. The largest error of the approach, 6.3 cm at $t = 5.1$ s, occurred in this turn, right after the highest speed of the flight. During the approach, the drone never came closer than 31.6 cm to the reconstruction, since its largest errors happened in open space between the boxes.

The speed lag shows in the speed plot of Fig. 7. The measured speed trailed the command while accelerating and stayed above it while decelerating toward the goal. These effects partly cancel, so the time to the goal stayed close to the plan, but at the end of the path this may have caused the overshoot. This 14.9 cm overshoot is the largest error of the whole run, and the divergence also peaked here, at 0.70 m/s, as the commanded velocity reversed to bring the drone back. The feedback then pulled the drone back onto the goal where it stayed within 5 cm of the goal from $t = 7.1$ s,

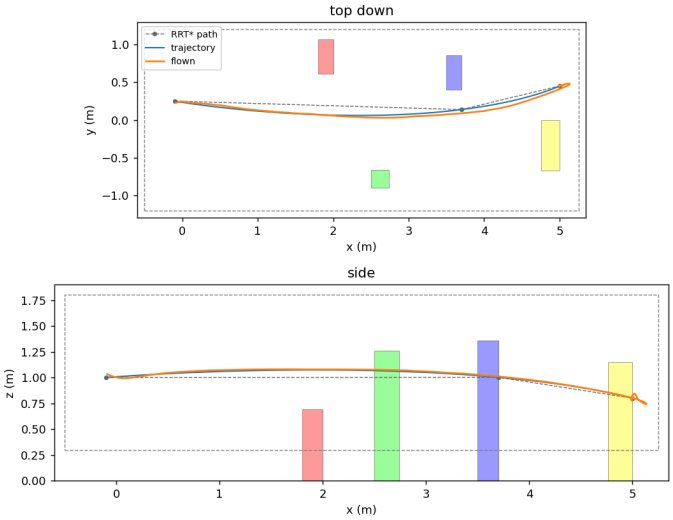


Fig. 6: Top (upper) and side (lower) views of flight 11, with the boxes and the flyable boundary (dashed). The RRT* path is dashed gray, the planned trajectory blue, and the flown path orange.

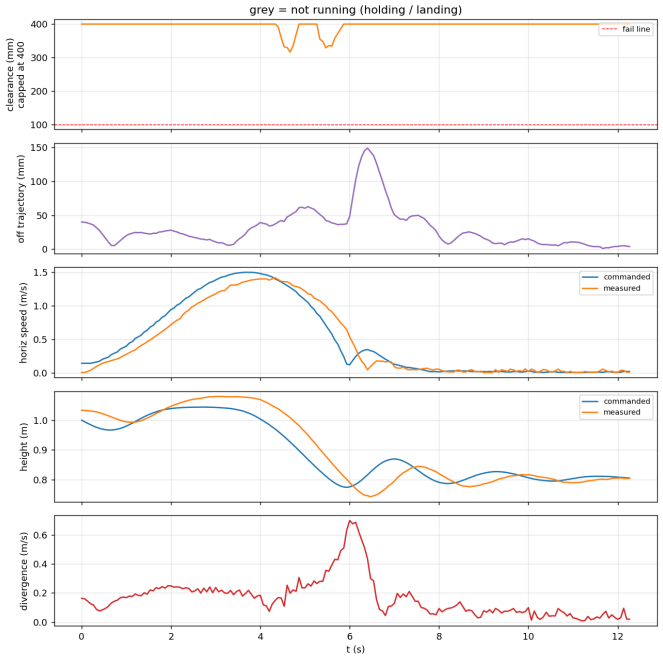


Fig. 7: Flight 11 over the whole run. From top: clearance to the reconstruction (capped at 400 mm, with the 100 mm collision limit), distance from the planned trajectory, commanded and measured horizontal speed, commanded and measured height, and divergence between commanded and actual velocity.

within 3 cm from $t = 7.9$ s, and ended 0.9 cm away. The commanded height in Fig. 7 shows a damped oscillation of several centimeters during this settling. The overshoot carried the drone toward the end wall of the volume, to within about 1.5 cm of the boundary shrunk by the 0.10 m margin, so the margin was nearly used up there, even though the drone stayed well clear of every box.

VIII. Extra Credit Videos

We are submitting the extra credit videos, VidTop.mp4 and VidObl.mp4, which show the same animation from a top-down and an oblique view, drawn over a render of the Gaussian splat. The code used to generate these videos was developed with the help of Claude Opus 5.5 [5]. Each video walks through the entire pipeline for hardware flight 11. It first shows the RRT* tree growing from the start, with rejected extensions marked, and the path to the goal straightening out over time as rewiring shortens it. It then shows the greedy shortcutting step checking straight lines between waypoints and reducing the tree path to the simplified waypoints, followed by the spline trajectory fitted through them. Finally, it plots the drone's position in real time from the Vicon pose recorded in the run log, on top of the planned trajectory. The trajectory and the flown path are taken directly from the logs of flight 11.

IX. Conclusion

This project utilized an RRT* planner, a quintic spline trajectory generator, and a tracking controller to navigate a quadrotor through a known 3D environment, first in a physics simulator and then on a real Crazyflie. In simulation, a tuned cascaded PID controller reached the goal on every trial without colliding with the environment, with a mean tracking error of 14 cm. Rewiring in RRT* combined with greedy shortcutting produced short paths with only a handful of waypoints, and the quintic spline turned these into a trajectory that is continuous up to snap.

The same planner and trajectory generator, with a smaller step size for the smaller volume, were then used on hardware. Because the Crazyflie closes its own velocity loop, the cascaded controller was reduced to an outer position loop, and a pure pursuit policy with velocity feedforward and a feedback term perpendicular to the path flew the Crazyflie through the boxes. With the most aggressive gain, $K_p = 2.5$, it reached the goal in 5.8 s at up to 1.42 m/s, stayed within 2.9 cm of the planned path on average and 6.3 cm at worst during the approach, and kept at least 31.6 cm from the reconstructed obstacles. The main sim-to-real effect was the drone's lag in reaching the commanded velocity, caused by drag, thrust error, and the time needed to tilt, which the simulated drone does not have. This divergence averaged 0.21 m/s during the approach, rose while the drone accelerated and braked, and dropped below 0.1 m/s at cruise speed. While the drone followed the path, the feedback term kept the position error to a few centimeters despite this lag, but it made the drone swing wide in the final turn, and because the drone could not stop as quickly as the plan asked, it overshot the goal by 14.9 cm, with the divergence peaking at 0.70 m/s, before settling to within 0.9 cm. Relative to the space available, the hardware tracking was similar to simulation, with the mean error using about a third of the inflation margin in both.

Acknowledgment

The authors would like to thank Prof. Nitin J. Sanket and TA Chaitanya Gaddipati for the opportunity to implement this project as a part of the RBE595-F02 Hands-on Autonomous Aerial Robotics course at WPI, and for the guidance that was provided along the way.

References

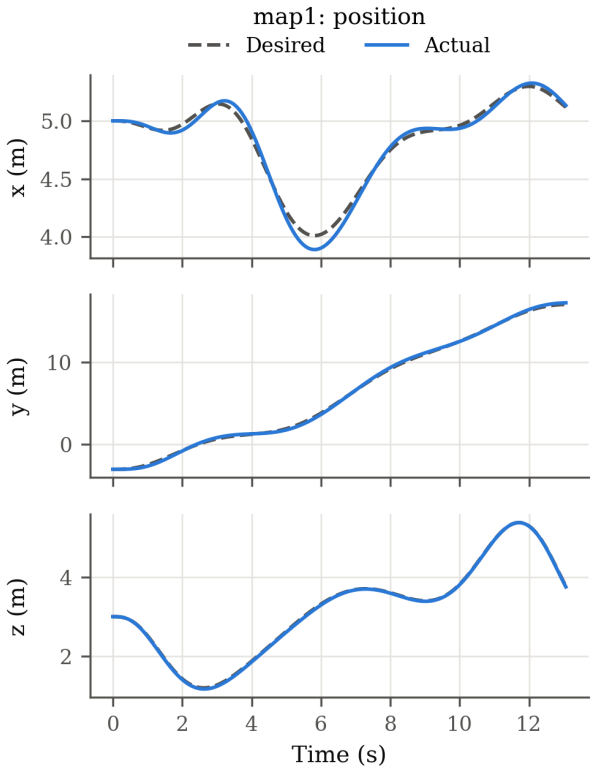
[1] C. Ericson, *Real-Time Collision Detection*. San Francisco, CA, USA: Morgan Kaufmann, 2005.

[2] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Computer Science Dept., Iowa State Univ., Ames, IA, USA, Tech. Rep. TR 98-11, 1998.

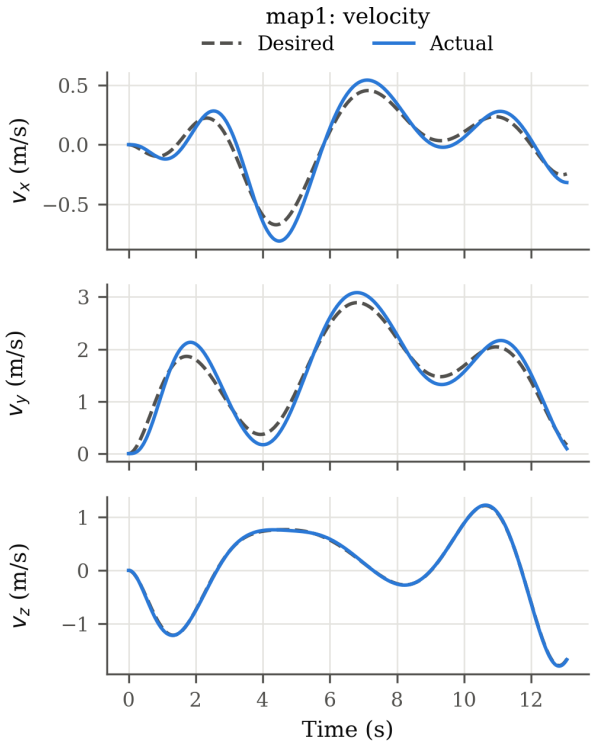
[3] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.

[4] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Robotics Institute, Carnegie Mellon Univ., Pittsburgh, PA, USA, Tech. Rep. CMU-RI-TR-92-01, 1992.

[5] Anthropic, "Claude Opus 5.5," [Online]. Available: https://www.anthropic.com/claude-opus-5-5.

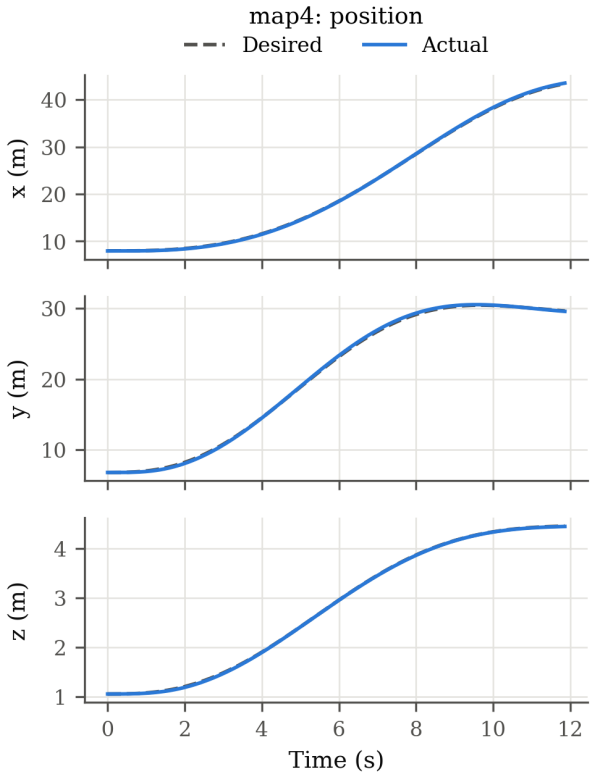


(a) Position

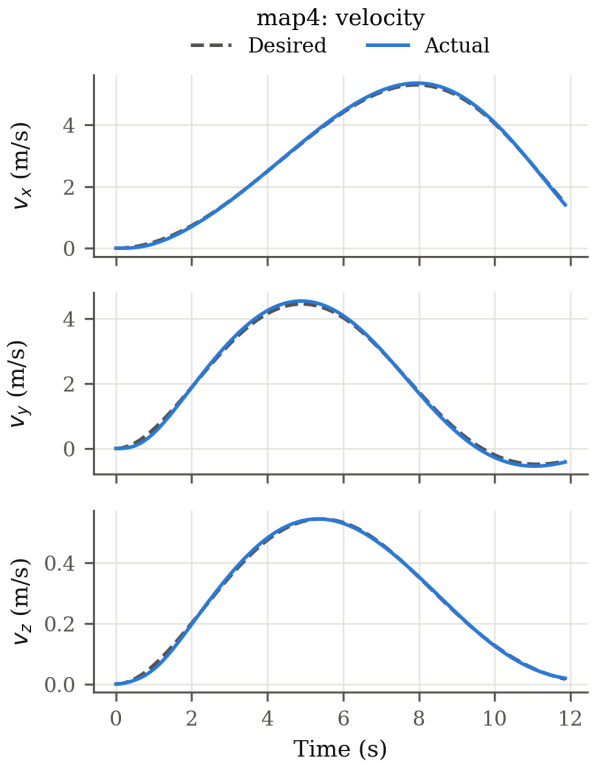


(b) Velocity

Fig. 8: Desired and actual position and velocity on Map 1.

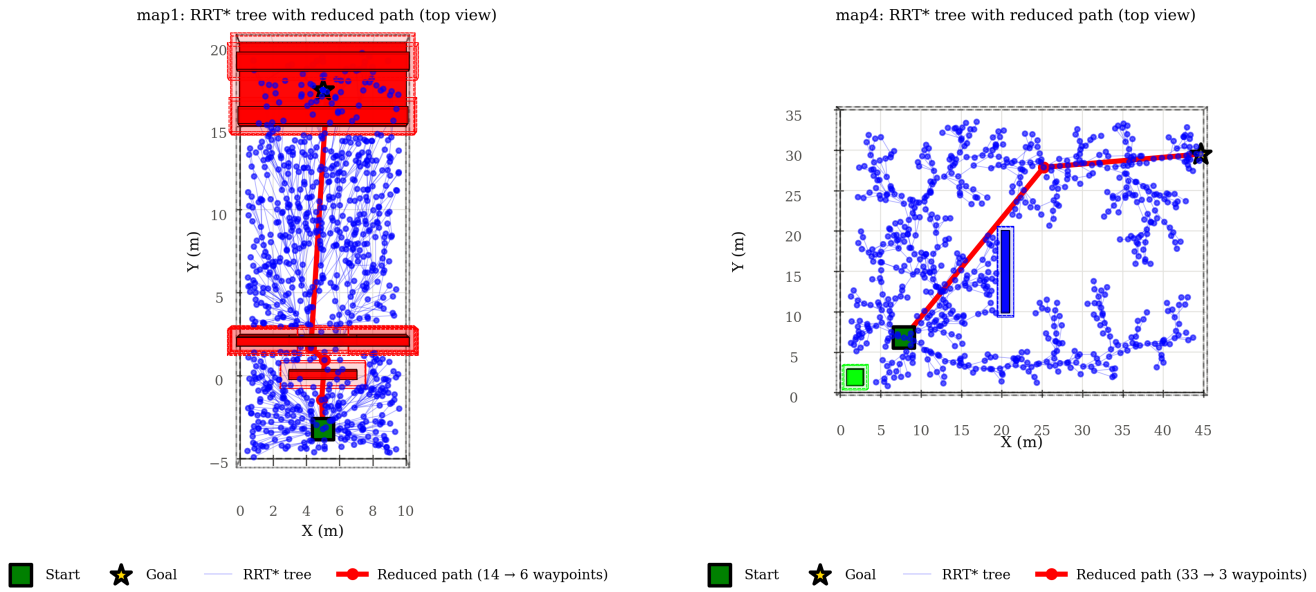


(a) Position



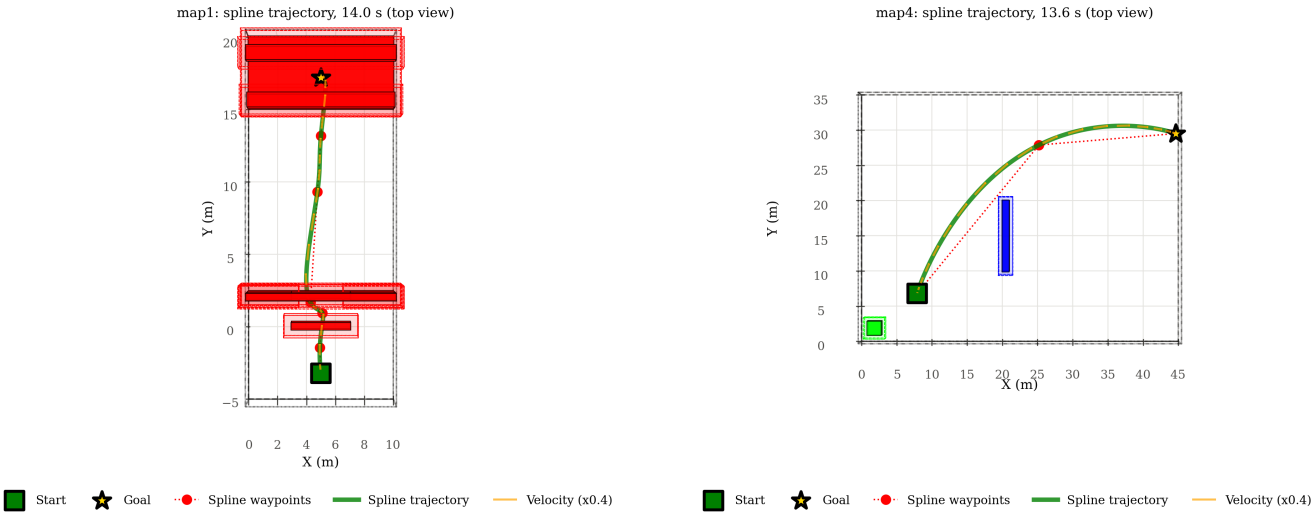
(b) Velocity

Fig. 9: Desired and actual position and velocity on Map 4.



(a) Map 1 (b) Map 4

Fig. 10: Top view of the simplified waypoints on Map 1 and Map 4.



(a) Map 1 (b) Map 4

Fig. 11: Top view of the simplified waypoints and spline trajectory on Map 1 and Map 4.

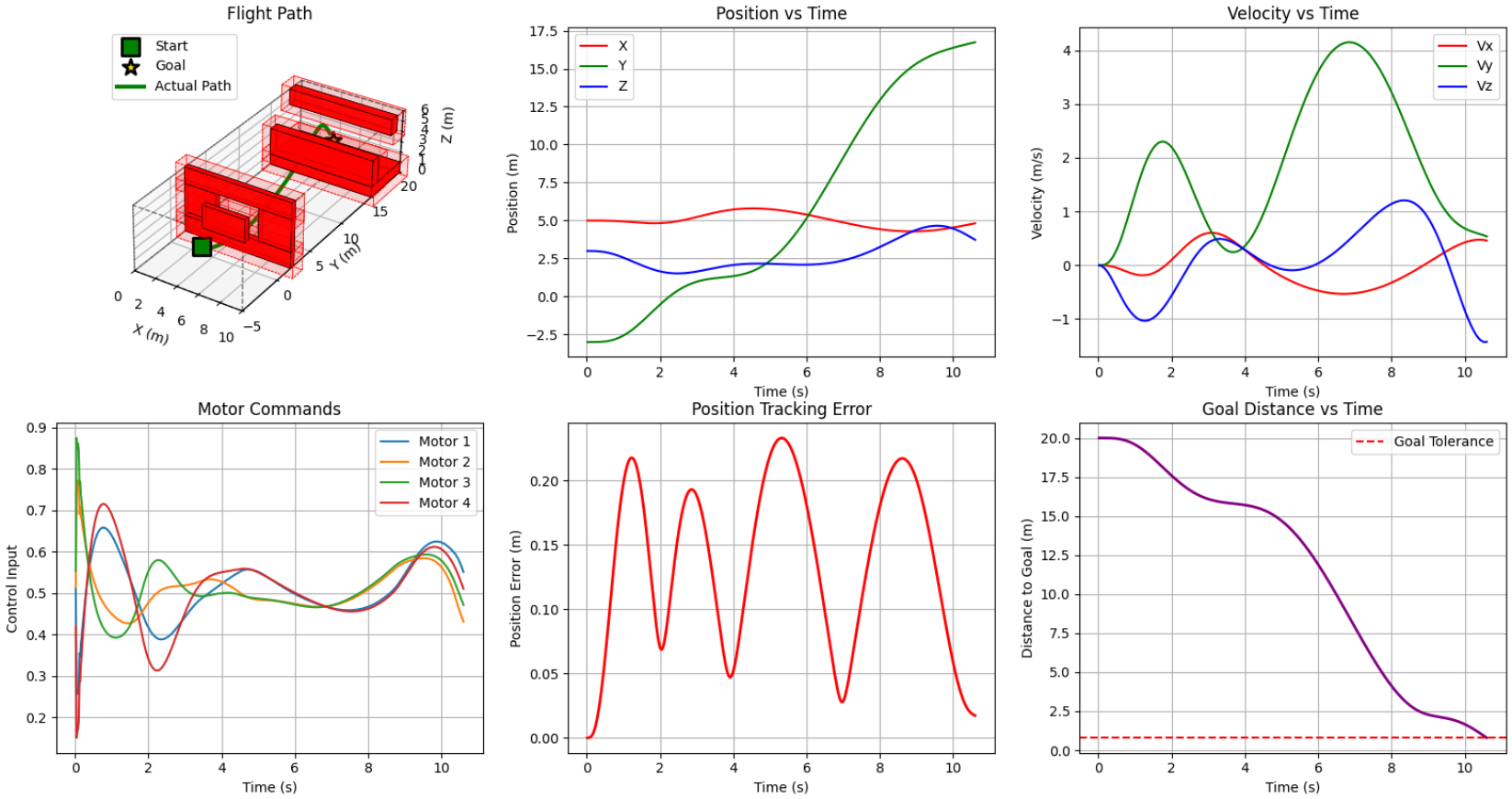


Fig. 12: Map 1 flight path and trajectory tracking results.

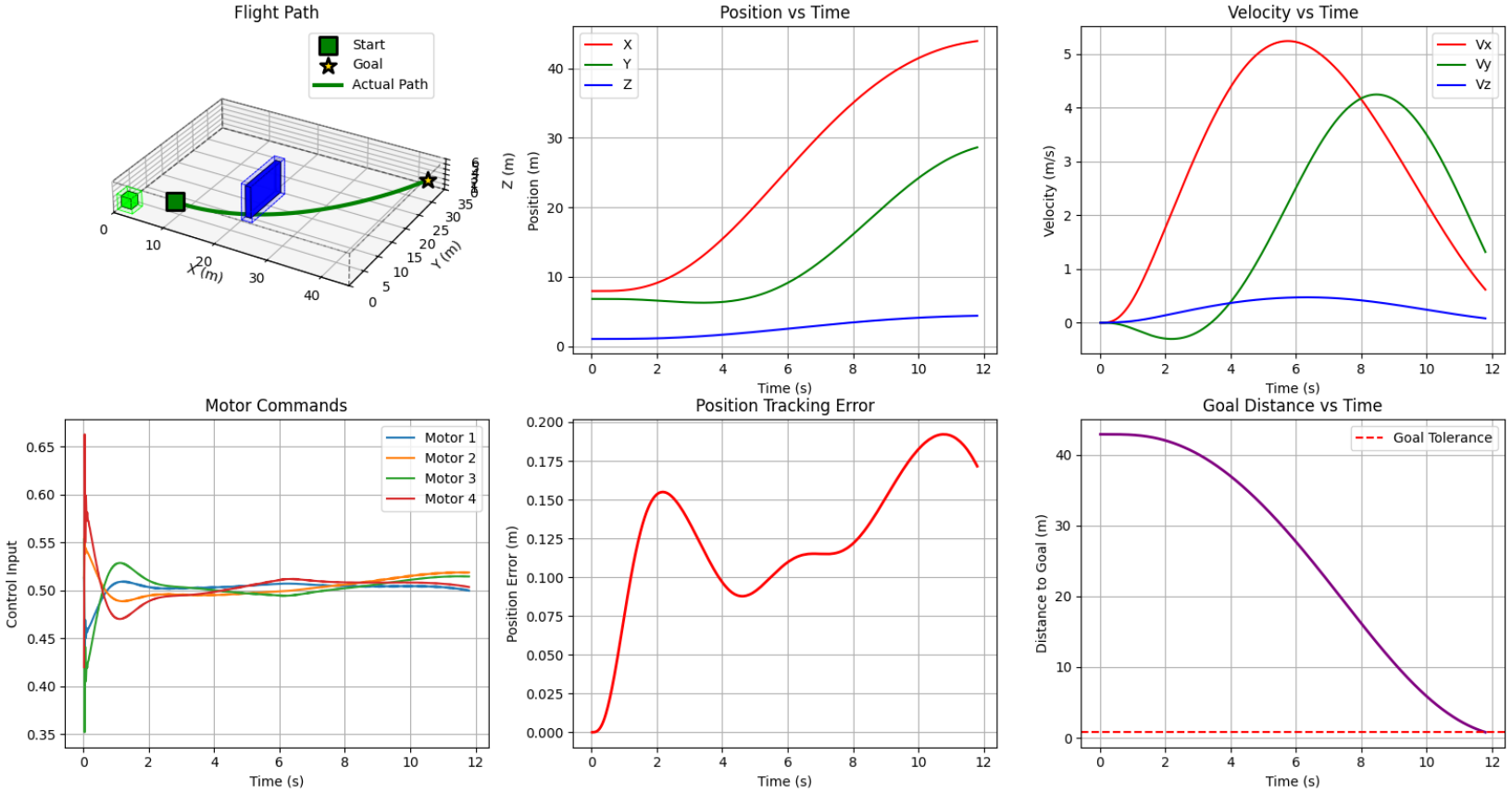


Fig. 13: Map 4 flight path and trajectory tracking results.